

Spring Framework / Spring Boot

Лекция 6 — курс «Kotlin», 1 курс СПбГУ МКН

90 минут · ~50 слайдов · Kotlin 2.3.x · Spring Boot 4.0 · 2026



Один кадр

60%

JVM-вакансий упоминают Spring
≈ StackOverflow Developer Survey 2024 (frameworks)

~75k 

GitHub `spring-projects/spring-boot`
github.com на 2026 (публично)

~10M/мес

запусков Spring Initializr
≈ доклады VMware Tanzu / Spring One 2024

23 года

возраст Spring (2003 → 2026)
Spring Framework 1.0 — март 2004; первые коммиты — 2003

За 90 минут вы поймёте, почему.

Кто говорит и о чём

План лекции

- Зачем фреймворк? — мотивация
- **IoC + DI** — главный концепт
- **Spring Boot** — что меняет всё
- **REST на Kotlin**: контроллеры, валидация, ошибки
- **Spring Data** + транзакции
- Профили, Actuator, тесты
- Что нового в 2026 (Boot 4 / Framework 7)

Что предполагается

- Знание базового Kotlin (класс, data class, `val / var`, расширения)
- Из прошлых лекций: HTTP, корутины, **Ktor** — для сравнения
- JVM на уровне «main, classpath, jar» — достаточно
- Никакой `web.xml` я от вас не жду

😬 Если на слове `@Bean` у вас выработался рефлекс улыбаться — значит, лекция уже наполовину удалась.

Карта лекции



I. Зачем

мотивация, экосистема,
цифры



II. IoC / DI

контейнер, бины, scope



III. Spring Boot

starters, autoconfig,
embedded



IV. Web / REST

@RestController,
validation, errors



V. Data

JPA, JdbcClient,
@Transactional



VI. Готово к prod

profiles, actuator, тесты



VII. 2026

Boot 4 · Framework 7 ·
Kotlin 2.3



Q&A

15 минут на вопросы



Часть I

Зачем вообще фреймворк?

Мир без фреймворка

Хочется простой HTTP-сервис: «GET /users/42 → JSON».

Что нужно собрать руками:

- HTTP-сервер (Tomcat? Netty?
`HttpServer.create()`)
- роутинг
- (де)сериализация JSON
- пул потоков
- логирование
- доступ к БД, пул соединений
- транзакции
`(Connection.setAutoCommit(false))`
- конфигурация (env vars + файлы + дефолты)
- метрики, healthcheck, graceful shutdown

```
fun main() {  
    val server = HttpServer.create(InetSocketAddress(8080)  
    val pool   = Executors.newFixedThreadPool(64)  
    val ds     = HikariDataSource(/* 12 параметров */)  
    val mapper = jacksonObjectMapper()  
    val users  = UserRepository(ds, mapper)  
    val orders = OrderRepository(ds, mapper)  
    val mail   = SmtMailer(/* host, port, tls, user, pas  
    val tx     = TransactionManager(ds)  
  
    server.executor = pool  
    server.createContext("/users") { ex →  
        // ручной парсинг path, метода, тела...  
        // ручная обработка ошибок...  
        // ручной commit/rollback...  
    }  
    server.start()  
}
```

😊 Это работает. Один раз. Для одного эндпоинта. Без авторизации.

Что такое Spring

Один из старейших и крупнейших фреймворков для JVM. Создан Родом Джонсоном в 2003 как реакция на боль EJB 2.

Краткая история

- **2003** — Spring 1.0: контейнер + DI + AOP вместо EJB
- **2014** — Spring Boot 1.0: «convention over configuration»
- **2017** — реактивный стек (WebFlux + Reactor)
- **2022** — Spring 6 / Boot 3: Jakarta EE, JDK 17 baseline, native (GraalVM)
- **окт 2025** — **Spring Boot 4.0** + **Framework 7**: JDK 25, structured logging, API versioning, корутинный context propagation

Что Spring даёт

- **IoC-контейнер**: жизненный цикл объектов, граф зависимостей
- **AOP**: транзакции, кэш, security как «срезы», а не как if'ы в коде
- **Web**: MVC + WebFlux + REST/HTTP clients
- **Data**: JPA / JDBC / Mongo / Redis под одним API
- **Boot**: конфигурация, embedded server, стартеры, Actuator
- сотни модулей: Security, Cloud, Batch, Integration, Kafka, Pulsar...

Spring — это не один проект

Spring Framework

core / context / beans, AOP, MVC, WebFlux

Spring Boot

starters, autoconfiguration, Actuator, embedded server

Spring Data

JPA, JDBC, Mongo, Redis, Cassandra
— один контракт

Spring Security

auth, OAuth2, JWT, RBAC, CSRF, фильтры

Spring Cloud

микросервисный тулинг: config, discovery, circuit breaker

Spring Batch

job/step, retry/skip, chunk-oriented обработка

Spring Integration

enterprise integration patterns, channels, adapters

Spring AI

LLM API, vector stores, prompt templates (2024+)

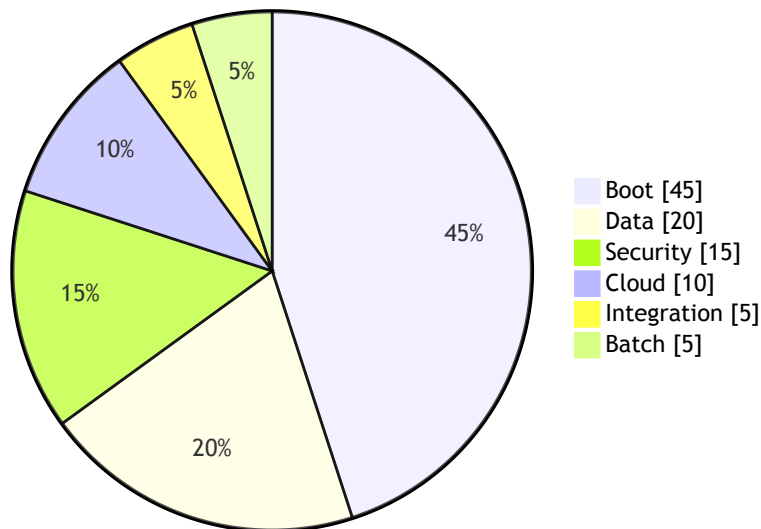
Spring Kafka / Pulsar

listeners, producers, transactional messaging

Сегодняшняя лекция = ядро (Framework + Boot + чуть Data). Всё остальное — за рамками.

Spring Universe — веса модулей

Spring Universe (по популярности модулей)



Ориентировочное распределение по числу starter'ов и популярности на GitHub. Точных цифр никто не публикует — принимать как «порядок величин».

В цифрах

Сколько Spring в JVM-мире

- ~**60–70%** Java/Kotlin backend-вакансий упоминают Spring
- **start.spring.io** — миллионы сгенерированных проектов в год
- Spring Initializr собирает проект быстрее, чем студент придумает имя пакету
- Поддержка от **Broadcom / VMware (Pivotal)**, релиз-цикл — 6 месяцев major-фич

Версии в 2026

- **Spring Framework 7.0** — март 2026
- **Spring Boot 4.0** — октябрь 2025
- **JDK baseline:** 17, рекомендован **JDK 25 LTS**
- **Kotlin baseline:** 2.2, в 2026 — **Kotlin 2.3.x**
- **Jakarta EE 11**, Servlet 6.1 (Tomcat 11)

«Spring Framework 7.0 retains a JDK 17 baseline while at the same time recommending JDK 25 as the latest LTS release. It also introduces a Jakarta EE 11 baseline and embraces Kotlin 2.2 as well as GraalVM 25.»

— Spring Framework 7.0 Release Notes (GitHub Wiki, 2026)

Три идеи, которые меняют всё

1. IoC

Не ты создаёшь зависимости — контейнер передаёт их тебе. Слабая связанность, тестируемость, замена реализации.

2. Convention over configuration

Boot догадывается о 90% — кодом подтверждаешь только то, что отличается. `application.yml` + аннотации.

3. AOP / cross-cutting concerns

Транзакции, security, логирование — **декларативно**, через аннотации на методах. Не лапша из `try/finally`.

😊 Java придумала фреймворки. Spring придумал, как ими пользоваться без слёз. А Spring Boot — как пользоваться Spring без `web.xml`.



Часть II

IoC и Dependency Injection

Проблема: связанность

```
class OrderService {  
    private val repo    = OrderRepository(DataSourceFactory.create())  
    private val mailer  = SmtпMailer("smtp.uni.ru", 25)  
    private val logger  = LoggerFactory.getLogger(javaClass)  
    private val clock   = Clock.systemUTC()  
  
    fun placeOrder(req: OrderRequest) { /* ... */ }  
}
```

Что не так:

- `OrderService` **знает**, как собрать каждого из своих коллабораторов
- В тестах `OrderRepository` нельзя подменить — конструктор идёт в реальную БД
- `SmtпMailer` → `MockMailer` — только через рефлексия или наследование
- 10-я зависимость → 10 правок в 10 местах



«Мы все взаимосвязаны» — звучит мило, но в архитектуре это диагноз.

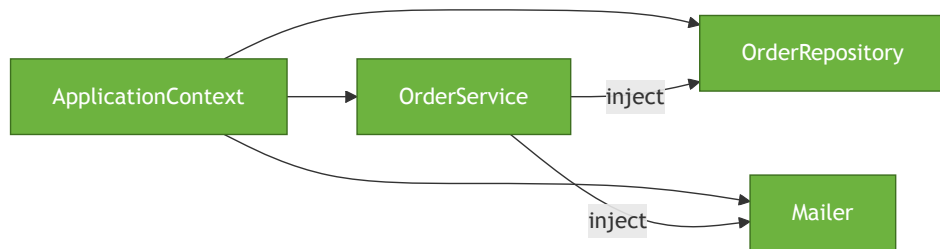
Идея IoC

Inversion of Control — контроль над созданием зависимостей передаётся **внешнему контейнеру**.

- Класс **объявляет**, что ему нужно (через конструктор)
- Кто и как это создаст — не его дело
- Контейнер видит весь граф и собирает его за один проход

Аналогия

- *Старый мир*: ты идёшь в магазин, покупаешь продукты, готовишь.
- *Spring*: ты заказываешь в ресторане. Что внутри — забота повара. Можешь даже не знать, что салат — из 12 ингредиентов.



Контейнер строит граф один раз на старте; дальше каждый bean — singleton по умолчанию.

Связанность: без DI vs. с DI

	Без DI	С DI
Точки правки при новой зависимости	$O(N^2)$ — «new» в каждом сборочном месте	$O(1)$ — параметр в конструкторе
Тестируемость	моки на каждом уровне через наследование / рефлексию	<code>@MockitoBean</code> / конструктор с fake
Onboarding нового разраба	«найди где <code>new</code> , замени»	«добавь параметр в конструктор»
Сборка графа зависимостей	вручную в <code>main()</code> или фабриках	контейнер решает за вас по topo-sort

«Оценка» по опыту миграции enterprise-кодбаз (Spring blog, Baeldung case studies, доклады SpringOne).

DI: три способа

Constructor injection

```
@Service
class OrderService(
    private val repo: OrderReposi
    private val mailer: Mailer,
)
```

- неизменяемые поля (`val`)
- естественно для Kotlin
- невозможно создать в неполном состоянии

Setter injection

```
@Service
class OrderService {
    @Autowired
    lateinit var repo: OrderRepos
}
```

- для optional-зависимостей
- разрешает циклы (но это смелл)

Field injection

```
@Service
class OrderService {
    @Autowired
    private lateinit var repo: Or
}
```

- кратко, но **скрывает** зависимости
- мешает unit-тестам без контейнера
- Spring сам не рекомендует

В Spring идиоматично: **конструктор**. В Kotlin — особенно: `primary constructor + val` = `immutable + auto-injectable`.

Стереотипы Spring

Аннотации, которые превращают класс в bean:

Аннотация	Семантика
<code>@Component</code>	базовый bean, общий случай
<code>@Service</code>	бизнес-логика (синоним <code>@Component</code> + смысл)
<code>@Repository</code>	доступ к данным; транслирует SQL-исключения в <code>DataAccessException</code>
<code>@Controller</code>	веб-контроллер (возвращает view)

```
package com.example.shop

@Service
class OrderService(
    private val repo: OrderRepository,
    private val mailer: Mailer,
) {
    fun place(req: OrderRequest): Order = TODO()
}

@Repository
class OrderRepository(
    private val jdbc: JdbcClient,
) {
    fun save(o: Order): Order = TODO()
}

@RestController
@RequestMapping("/api/orders")
class OrderController(
    private val service: OrderService,
) {
    @PostMapping
    fun create(@RequestBody req: OrderRequest) =
        service.place(req)
}
```

@Bean — фабрика бинов

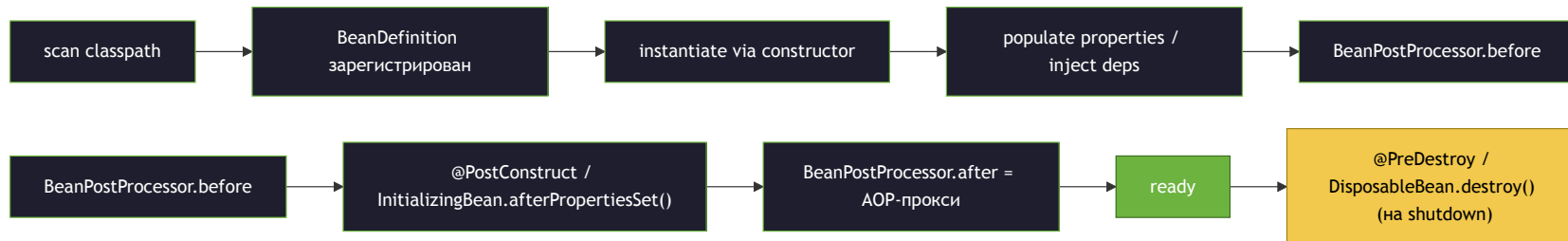
Когда класс **не наш** (нельзя поставить аннотацию), или нужна нетривиальная сборка — используем `@Configuration` + `@Bean`:

```
@Configuration
class HttpClients(
    private val props: ClientProps,
) {
    @Bean
    fun objectMapper(): ObjectMapper =
        jsonMapper {
            addModule(kotlinModule())
            addModule(JavaTimeModule())
            disable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS)
        }

    @Bean
    fun paymentRestClient(builder: RestClient.Builder): RestClient =
        builder
            .baseUrl(props.paymentUrl)
            .defaultHeader("X-Api-Key", props.apiKey)
            .build()
}
```

Параметры `@Bean`-метода — это тоже инъекция. `builder` сюда придёт уже сконфигурированным Boot'ом.

Жизненный цикл bean'a



Полезные хуки: `@PostConstruct` для инициализации (после инъекции), `@PreDestroy` для аккуратного shutdown'a (закрыть пул, отписаться, сбросить буферы).

Scope: сколько экземпляров?

Scope	Когда
<code>singleton</code> <i>(default)</i>	один на ApplicationContext — 95% случаев
<code>prototype</code>	новый экземпляр при каждом <code>getBean</code>
<code>request</code> (web)	по одному на HTTP-запрос
<code>session</code> (web)	по одному на сессию пользователя

```
@Service
@Scope(BeanDefinition.SCOPE_PROTOTYPE)
class ReportBuilder { /* stateful, не singleton */ }
```

 Spring singleton — это **не** GoF Singleton. Один экземпляр *в этом контейнере*; два контекста — два экземпляра.

Чем IoC не является

НЕ магия

- Контейнер — обычный объект
(`AnnotationConfigApplicationContext`)
- На старте сканирует classpath, парсит аннотации, строит граф
- Никаких `__metaclass__` и hidden hooks — есть стек, есть исходники

НЕ только reflection

- Boot 3+ умеет **AOT** (ahead-of-time): bean-definitions генерируются на этапе сборки
- GraalVM native-image работает с Boot, потому что граф известен заранее

НЕ Service Locator

- Service Locator: объект *сам* запрашивает зависимость у глобального реестра
- IoC: зависимости передаются **извне**, объект не знает контейнер
- Тест: можно ли создать класс `new OrderService(repo, mailer)` без Spring? Если да — это DI.

НЕ "только при старте"

- Контекст можно перезапускать (DevTools, `@RefreshScope`)
- Конфиги горячо обновляются через Spring Cloud Config

«Inversion of Control is the principle. Dependency Injection is one of the patterns implementing this principle.»
— Spring Framework Reference Documentation



Часть III

Spring Boot — что меняет всё

Do Boot

web.xml :

```
<servlet>
  <servlet-name>app</servlet-name>
  <servlet-class>org.springframework.web.servlet.Dispatcher
  <init-param>
    <param-name>contextConfigLocation</param-name>
    <param-value>/WEB-INF/spring/dispatcher.xml</param-value>
  </init-param>
  <load-on-startup>1</load-on-startup>
</servlet>
```

applicationContext.xml :

```
<bean id="dataSource" class="...">
  <property name="url" value="..." />
  <property name="username" value="..." />
  <!-- 100 строк про hibernate, transactions, jpa -->
</bean>
```

Co Spring Boot

Application.kt :

```
@SpringBootApplication
class ShopApplication

fun main(args: Array<String>) {
  runApplication<ShopApplication>(*args)
}
```

application.yml :

```
server:
  port: 8080
spring:
  datasource:
    url: jdbc:postgresql://localhost/shop
    username: shop
    password: ${DB_PASSWORD}
```



R.I.P. web.xml . 1999–2014.

Lines of XML config: 2008 → 2025



Toy todo-app, измерение приблизительное — сравниваемся по референсным гайдам эпох. Из ~150 строк XML осталось 0.

4 столпа Spring Boot

Starters

Транзитивные зависимости пакетом: один `spring-boot-starter-webmvc` = Tomcat + MVC + Jackson + validation

Auto-configuration

Видит JDBC на classpath → конфигурирует DataSource.
`@ConditionalOnClass` повсюду.

Embedded server

Tomcat / Jetty встроен в JAR. `java -jar app.jar` — и поехали. Никакого app-сервера.

Externalized config

`application.yml`, env vars, command-line — единый property resolver с приоритетами.

Бонус: Actuator

Один starter — десяток endpoint'ов: `/health`, `/info`, `/metrics`, `/loggers`, `/env`. Готовая observability.

«Spring Boot 4.0 has a new modular design and now ships smaller focused modules rather than several large jars.»

— Spring Boot 4.0 Release Notes

Spring Initializr — start.spring.io

Веб-форма на 30 секунд:

1. **Project:** Gradle-Kotlin
2. **Language:** Kotlin
3. **Spring Boot:** 4.0.x
4. **Dependencies:** Web, Data JPA, PostgreSQL, Validation, Actuator
5. **Generate** → zip с готовым проектом

Что внутри

- `build.gradle.kts` с правильными версиями
- `Application.kt` с `@SpringBootApplication`
- `application.properties` (пустой)
- структура пакетов
- `.gitignore`, `mvnw` / `gradlew`, тесты

Альтернатива

- IntelliJ IDEA: *New* → *Project* → *Spring Boot* — тот же Initializr внутри
- CLI: `spring init --dependencies=webmvc,data-jpa,postgresql shop`
- IntelliJ Expylt-плагин: ещё и подсказывает по бинам

Минимальный Boot-app

```
package com.example.shop

@SpringBootApplication
class ShopApplication

fun main(args: Array<String>) {
    runApplication<ShopApplication>(*args)
}

@RestController
class HelloController {
    @GetMapping("/hello")
    fun hello(): Map<String, String> = mapOf("msg" to "Hello, MKH!")
}
```

```
./gradlew bootRun
# → Tomcat started on port 8080 · curl localhost:8080/hello → {"msg":"Hello, MKH!"}
```

@SpringBootApplication = @Configuration + @EnableAutoConfiguration + @ComponentScan .

Autoconfiguration: как это работает

```
@AutoConfiguration
@ConditionalOnClass(DataSource::class)
@ConditionalOnMissingBean(DataSource::class)
@EnableConfigurationProperties(DataSourceProperties::class)
class DataSourceAutoConfiguration {
    @Bean
    fun dataSource(props: DataSourceProperties): DataSource =
        HikariDataSource().apply {
            jdbcUrl = props.url; username = props.username; password = props.password
        }
}
```

1. `@ConditionalOnClass` — `DataSource` есть в classpath (приехал со starter'ом)
2. `@ConditionalOnMissingBean` — пользователь сам не объявил `DataSource`
3. `@EnableConfigurationProperties` — `spring.datasource.*` → `DataSourceProperties`

Не магия — **условные аннотации**. Публикуется в `META-INF/spring/.../AutoConfiguration.imports`.

Properties — типобезопасно

application.yml :

```
shop:
  payment:
    url: https://pay.example.com
    api-key: ${PAYMENT_KEY}
    timeout: 5s
    retries: 3
```

PaymentProps.kt :

```
@ConfigurationProperties(prefix = "shop.payment")
data class PaymentProps(
    val url: String,
    val apiKey: String,
    val timeout: Duration = Duration.ofSeconds(5),
    val retries: Int = 3,
)
```

✓ Kotlin `data class` — immutable, null-safe, валидация при старте. Опечатка в YAML → приложение не стартует.

Это не магия

Поднимаем капот autoconfiguration. Адрес «волшебного ящика» — файл в jar:

```
META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports
```

```
@AutoConfiguration
@ConditionalOnClass(DataSource::class)
@ConditionalOnMissingBean(DataSource::class)
@EnableConfigurationProperties(DataSourceProperties::class)
class DataSourceAutoConfiguration {
    @Bean fun dataSource(p: DataSourceProperties): DataSource = HikariDataSource()
}
```

«Auto-configuration classes are regular `@Configuration` classes that are typically applied through condition annotations such as `@ConditionalOnClass` or `@ConditionalOnMissingBean` .»

— Spring Boot Reference, «Creating Your Own Auto-configuration»



Магия = условные бины + порядок + конвенции. Никаких джиннов.



Часть IV

Web и REST на Kotlin

Spring MVC vs WebFlux

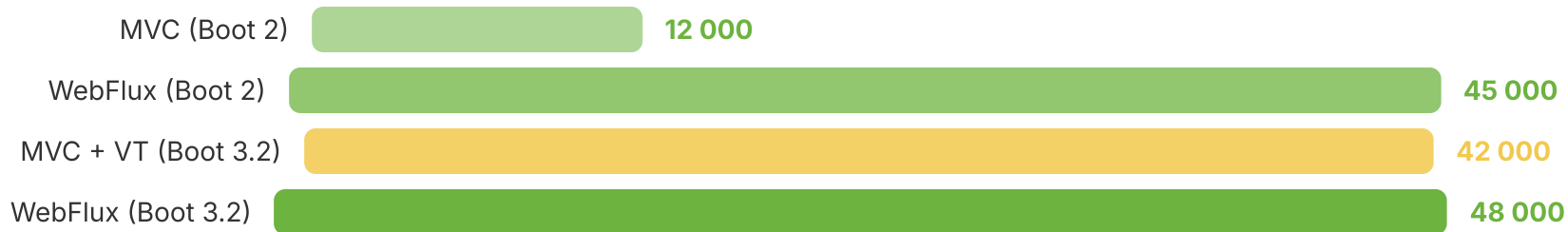
	Spring MVC	Spring WebFlux
Модель	thread-per-request, blocking	event loop, non-blocking, reactive
API	servlet (<code>HttpServletRequest</code>)	<code>ServerHttpRequest</code> , <code>Reactor</code>
Стиль кода	<code>fun foo(): User</code>	<code>fun foo(): Mono<User></code> или <code>suspend fun</code>
Контейнер	Tomcat / Jetty	Netty / Tomcat 11
Выбирать когда	стандартный CRUD; есть JDBC	стримы, SSE, тысячи долгих коннектов

В 2026 — обычно MVC

- **Virtual threads** в JDK 21+ → блокирующий стиль масштабируется
- Boot 4 + `spring.threads.virtual.enabled=true` → каждый запрос на VT
- WebFlux нужен **специфично** — там, где реально нет блокирующего варианта

MVC vs WebFlux после Virtual Threads

Throughput на 10k коннектов (RPS, условно)



«До Loom WebFlux был быстрее в 3–4 раза. После — паритет.»

По публичным бенчам TechEmpower / Spring blog 2024 — порядок величин.

@RestController за 8 строк

```
@RestController
@RequestMapping("/api/users")
class UserController(private val service: UserService) {
    @GetMapping("/{id}")
    fun get(@PathVariable id: Long): UserDto = service.findById(id)
}
```

И всё. Что Boot делает за вас:

- маршрутизация GET /api/users/42 + parsing {id} → Long
- сериализация UserDto в JSON через Jackson, Content-Type и 200 OK
- EntityNotFoundException → 404 через @ControllerAdvice

В Boot 4 — Jackson 3.x по умолчанию; можно переключиться на **kotlinx.serialization**.

CRUD-набор аннотаций

```
@RestController
@RequestMapping("/api/users")
class UserController(private val service: UserService) {

    @GetMapping
    fun list(@RequestParam(required = false) q: String?): List<UserDto> =
        service.search(q)

    @GetMapping("/{id}")
    fun get(@PathVariable id: Long): UserDto = service.findById(id)

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    fun create(@RequestBody @Valid req: CreateUserRequest): UserDto =
        service.create(req)

    @PutMapping("/{id}")
    fun update(
        @PathVariable id: Long,
        @RequestBody @Valid req: UpdateUserRequest,
    ): UserDto = service.update(id, req)

    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
```

Валидация

```
data class CreateUserRequest(  
    @field:NotBlank  
    @field:Size(min = 2, max = 64)  
    val name: String,  
  
    @field:Email  
    val email: String,  
  
    @field:Min(0)  
    @field:Max(150)  
    val age: Int,  
)
```

`@field:` — Kotlin-специфика: аннотация ставится **на поле**, а не на параметр конструктора. Иначе Bean Validation её не увидит.

Аннотации из `jakarta.validation.constraints.*`: `@NotBlank`, `@Email`, `@Size`, `@Min`, `@Max`, `@Pattern`, `@Past`, ...

```
@PostMapping  
fun create(  
    @RequestBody @Valid req: CreateUserRequest,  
): UserDto = service.create(req)
```

Что происходит при невалидном теле:

1. Spring валидирует `req` до входа в метод
2. На любую ошибку — `MethodArgumentNotValidException`
3. `@ControllerAdvice` (см. след. слайд) превращает в `400 Bad Request` с RFC 7807 `ProblemDetail`



`@Valid` — это пограничный контроль для DTO. Все, кто не прошёл, разворачиваются на 400, не доходя до бизнес-логики.

Обработка ошибок: RFC 7807 ProblemDetail

```
@ControllerAdvice
class ApiExceptionHandler {
    @ExceptionHandler(EntityNotFoundException::class)
    fun handleNotFound(ex: EntityNotFoundException): ProblemDetail =
        ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND, ex.message ?: "not found")
            .apply { title = "Resource not found" }

    @ExceptionHandler(MethodArgumentNotValidException::class)
    fun handleValidation(ex: MethodArgumentNotValidException): ProblemDetail =
        ProblemDetail.forStatusAndDetail(HttpStatus.BAD_REQUEST, "Validation failed").apply {
            title = "Bad request"
            setProperty("errors", ex.bindingResult.fieldErrors.map {
                mapOf("field" to it.field, "message" to (it.defaultMessage ?: ""))
            })
        }
}
```

Ответ: { "type": "about:blank", "title": "Bad request", "status": 400, "detail": "Validation failed", "errors": [...] }

Spring vs Ktor: REST

```
// Spring MVC
@RestController
@RequestMapping("/api/users")
class UserController(
    private val service: UserService,
) {
    @GetMapping("/{id}")
    fun get(@PathVariable id: Long): UserDto =
        service.findById(id)

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    fun create(@RequestBody @Valid req: CreateUserRequest
        : UserDto = service.create(req)
}
```

```
// Ktor
fun Application.userRoutes(service: UserService) {
    routing {
        route("/api/users") {
            get("/{id}") {
                val id = call.parameters["id"]!!.toLong()
                call.respond(service.findById(id))
            }
            post {
                val req = call.receive<CreateUserRequest>
                // валидируем руками или через kotlinx-validation
                call.respond(HttpStatusCode.Created, serv
            }
        }
    }
}
```

Spring: декларативно (аннотации), плотный type-mapping, валидация бесплатно.

Ktor: императивно (DSL), меньше магии, ближе к тому, что реально происходит.

HTTP-клиенты в Spring

Spring 6/Boot 3+ — `RestClient` (синхронный, fluent):

```
@Component
class PaymentClient(private val client: RestClient) {
    fun charge(req: ChargeRequest): ChargeResponse =
        client.post().uri("/charge")
            .contentType(MediaType.APPLICATION_JSON)
            .body(req).retrieve()
            .body(ChargeResponse::class.java)!!
}
```

Boot 4 — **HTTP Service Clients** (декларативно, как Retrofit):

```
@HttpExchange(url = "/api/v1")
interface InventoryClient {
    @GetExchange("/items/{sku}")
    fun byId(@PathVariable sku: String): Item
}
```

Boot 4 автоконфигит HTTP Service Clients — интерфейс + аннотации, реализация генерится Spring'ом (Spring Boot 4.0 Release Notes).



Часть V

Spring Data и доступ к данным

Зачем Spring Data

Один контракт, десять реализаций:

Модуль	Бэкенд
<code>spring-data-jpa</code> / <code>-jdbc</code>	реляционные БД (JPA/Hibernate или простой ORM)
<code>spring-data-mongodb</code>	MongoDB
<code>spring-data-redis</code>	Redis
<code>spring-data-cassandra</code>	Cassandra
<code>spring-data-elasticsearch</code>	Elasticsearch

Везде — ****Repository-абстракция****: интерфейс с методами, реализацию пишет Spring.

 `findByEmailIgnoreCaseAndActiveTrue` — это **parser имени метода**, не магия. Запрос строится на старте; ошибки тоже.

JPA-репозиторий

```
@Entity
@Table(name = "users")
class UserEntity(
    @Id @GeneratedValue
    val id: Long? = null,
    val email: String,
    val name: String,
    val createdAt: Instant = Instant.now(),
)

interface UserRepository : JpaRepository<UserEntity, Long> {
    fun findByEmail(email: String): UserEntity?
    fun findByNameContainingIgnoreCase(part: String): List<UserEntity>

    @Query("select u from UserEntity u where u.createdAt > :since")
    fun recent(since: Instant): List<UserEntity>
}
```

Что бесплатно

save, findById, findAll, delete, paging, sorting, derived queries

Kotlin-нюанс

JPA требует пустой конструктор → плагин `kotlin-jpa` делает `open` + нет-аргов автоматически

JdbcClient — без ORM-магии (Boot 3.2+)

```
@Repository
class UserJdbcRepository(
    private val jdbc: JdbcClient,
) {
    fun findById(id: Long): UserEntity? =
        jdbc.sql("select id, email, name, created_at from users where id = :id")
            .param("id", id)
            .query(UserEntity::class.java)
            .optional()
            .orElse(null)

    fun save(u: UserEntity): Long =
        jdbc.sql("""
            insert into users(email, name, created_at)
            values (:email, :name, :createdAt)
            returning id
        """).trimIndent()
            .params(mapOf("email" to u.email, "name" to u.name, "createdAt" to u.createdAt))
            .query(Long::class.java)
            .single()
}
```

Когда выбирать `JdbcClient` вместо JPA: явный SQL, минимум магии, нужны точные планы запросов.

@Transactional

```
@Service
class OrderService(
    private val orders: OrderRepository,
    private val items: ItemRepository,
    private val mailer: Mailer,
) {

    @Transactional
    fun place(req: OrderRequest): Order {
        val order = orders.save(Order(req))
        items.reserve(order.id!!, req.items)    // если упадёт → rollback
        mailer.send(req.email, "Order placed") // вне транзакции по смыслу — выноси наружу
        return order
    }
}
```

Правила по умолчанию

- rollback на **unchecked** (`RuntimeException`)
- НЕ rollback на checked
- propagation `REQUIRED`
- isolation — из БД

Подводные камни

- работает только через **прокси**: вызов `this.method()` внутри класса **не** начнёт новую транзакцию
- в Kotlin: класс должен быть `open` — `kotlin-spring` плагин делает это автоматически
- не оборачивайте в неё долгие IO (mail, http)

Тёмная сторона: N+1, proxy hell, magic broken

1. N+1 problem

```
val users = userRepo.findAll()
users.map { it.orders.size }
// 1000 users → 1001 запрос
```

Решение: `@EntityGraph`, `JOIN FETCH` или явный `JdbcClient`.

2. Proxy hell

```
@Service
class A {
    fun outer() = this.inner() // Н
    @Transactional
    fun inner() { /* ... */ }
}
```

`@Transactional` self-invocation не работает — прокси видит только внешние вызовы.

3. Когда magic ломается

`@ConditionalOnClass` matched — стартёр включился. Но `spring.datasource.url` пуст → Hikari падает в runtime с невнятным `stacktrace`ом на 200 кадров.

Решение: `-debug` при старте → список сработавших и пропущенных conditions.

🤖 Spring уважает вас достаточно, чтобы не врать. Но и достаточно, чтобы не объяснять каждый раз.

Миграции схемы — Flyway

```
src/main/resources/db/migration/  
├─ V1__create_users.sql  
├─ V2__add_email_index.sql  
└─ V3__add_orders.sql
```

V1__create_users.sql :

```
create table users (  
    id          bigserial primary key,  
    email       varchar(255) not null unique,  
    name        varchar(255) not null,  
    created_at  timestamptz not null default now()  
);
```

Boot и Flyway

- positionner starter: `org.flywaydb:flyway-core`
- видит `db/migration` → ищет `V*__.sql`
- запускает миграции **до** инициализации Hibernate
- статус — в таблице `flyway_schema_history`

Альтернативы

- **Liquibase**: XML/YAML/JSON-changelog'и
- **Exposed migrations** (Kotlin-only)
- руками — категорически не надо



Часть VI

Профили, конфигурация, тесты

Профили: один JAR — N окружений

```
# application.yml
spring:
  profiles.active: ${ENV:dev}
shop:
  payment.url: https://pay-stage.example.com
```

```
---
spring.config.activate.on-profile: prod
shop:
  payment.url: https://pay.example.com
logging.level.root: WARN
```

```
---
spring.config.activate.on-profile: dev
shop:
  payment.url: http://localhost:9090
logging.level.com.example: DEBUG
```

```
@Service @Profile("!prod")
class FakeMailer : Mailer { /* не отправляет */ }
```


Spring Boot Actuator

```
management:
  endpoints:
    web:
      exposure:
        include: health, info, metrics, loggers, env
  endpoint:
    health:
      show-details: when_authorized
```

`/actuator/health`

UP/DOWN, проверка БД, Redis, диска

`/actuator/health/liveness`

для k8s probe — приложение живо

`/actuator/health/readiness`

k8s readiness — готово принимать трафик

`/actuator/metrics`

Micrometer + Prometheus exporter

`/actuator/info`

git commit, build version

`/actuator/loggers`

менять log level в рантайме

В Boot 4: **liveness** и **readiness probes** включены по умолчанию. Production-grade observability в одном starter'е.

Тестирование: пирамида

Unit (быстро)

```
class OrderServiceTest {  
    val repo = mockk<OrderRepository>  
    val service = OrderService(repo)  
  
    @Test fun `places order`() {  
        every { repo.save(any()) }  
        service.place(req()) shouldNotThrow  
    }  
}
```

без Spring — Kotlin/JUnit

Slice tests

```
@WebMvcTest(UserController::class)  
class UserControllerTest(  
    @Autowired val mvc: MockMvc,  
) {  
    @MockitoBean lateinit var svc: IOrderService  
  
    @Test fun `404 on missing`() {  
        every { svc.findById(any()) }  
        mvc.get("/api/users/42")  
            .andExpect { status { is404 } }  
    }  
}
```

только **web-слой**

@SpringBootTest + TC

```
@SpringBootTest  
@AutoConfigureMockMvc  
@Testcontainers  
class OrderE2ETest {  
    @Container companion object {  
        @JvmStatic val pg =  
            PostgreSQLContainer("postgres")  
    }  
    @Autowired lateinit var mvc: MockMvc  
}
```

долго, но честно

Boot 4: `@SpringBootTest` больше не даёт `MockMvc/WebClient` автоматически — нужны `@AutoConfigureMockMvc` ; `@MockBean` → `@MockitoBean` .

Test slices — целиком

Аннотация	Что поднимает
<code>@SpringBootTest</code>	весь контекст приложения
<code>@WebMvcTest</code>	DispatcherServlet, контроллеры, валидаторы — без сервисов и БД
<code>@DataJpaTest</code>	JPA-слой, in-memory H2 (или Testcontainers через <code>@AutoConfigureTestDatabase(NONE)</code>)
<code>@JsonTest</code>	только Jackson/kotlinx, проверяем (де)сериализацию DTO
<code>@RestClientTest</code>	HTTP-клиент + <code>MockRestServiceServer</code>

Принцип: чем меньше слой — тем быстрее тест. **Не** используйте `@SpringBootTest` там, где хватает `@WebMvcTest` .



«Тесты медленные, потому что Spring».



Часть VII

Что нового в Spring 2026

Spring Boot 4.0 (октябрь 2025) — главное

Платформа

- **JDK 17 baseline**, рекомендован **JDK 25 LTS**
- **Jakarta EE 11** + Servlet 6.1 (Tomcat 11, Jetty 12.1)
- **Kotlin 2.2.20+**
- Gradle 9 поддержан, **Undertow удалён**

Web / clients

- **HTTP Service Clients** auto-configuration
- **API Versioning** через `spring.mvc.apiversion.*`
- `spring-boot-starter-webmvc` (вместо `-web`)
- `RestTestClient` — non-reactive аналог `WebTestClient`
- Jackson 3.x default; **kotlinx.serialization** через свой стартер

Concurrency

- `spring.threads.virtual.enabled=true` теперь покрывает и **JDK HttpClient**
- **Liveness/readiness probes — by default**

Testing (breaking)

- `@MockBean` → `@MockitoBean`
- `@SpringBootTest` не даёт `MockMvc` автоматом → `@AutoConfigureMockMvc`

Модуляризация

- мелкозернистые `spring-boot-⟨technology⟩` модули
- старые «толстые» starters → `-classic` (мост на миграцию)

«Auto-configured HTTP clients backed by the JDK `HttpClient` are now configured to use virtual threading when `spring.threads.virtual.enabled` is `true` .»

— Spring Boot 4.0 Release Notes

Spring Framework 7.0 (март 2026)

Null safety первого класса

- Полный переход на **JSpecify** вместо JSR 305
- В Kotlin — может потребоваться править `? / !!`
- *blog post: «Null-safe applications with Spring Boot 4», ноя 2025*

Resilience в core

- `RetryTemplate` , `@Retryable` , `@ConcurrencyLimit`
- перенесены из Spring Retry в `spring-core`
- включается `@EnableResilientMethods`
- `@Retryable` сам адаптируется к реактивным методам

Прокси

- Все proxy processors (включая `@Async`) — **CGLIB по умолчанию**
- Opt-out: `@Proxyable(INTERFACES)`

HTTP

- `RestTemplate` — **deprecated в docs** (станет `@Deprecated` в 7.1)
- `OkHttp3` поддержка удалена
- `HttpHeaders` больше **не** наследует `MultiValueMap` (breaking)

AOT / Native

- GraalVM 25 + новый формат reachability metadata
- JEP 484 Class-File API при JDK 24+

Kotlin 2.3.x + Spring

```
@RestController
class UserController(private val service: UserService) {

    @GetMapping("/{id}")
    suspend fun get(@PathVariable id: Long): UserDto =
        service.findById(id)           // suspend — Spring сам снимет корутину
}
```

Что есть из коробки

- `suspend fun` в `@RestController` — Spring MVC и WebFlux умеют (через `CoroutinesUtils`)
- **kotlin-spring** плагин — автоматически делает классы `open` (нужно для AOP-проксей)
- **kotlin-jpa** плагин — no-arg конструктор + `open` для entities
- **Boot 4 starter** для **kotlinx.serialization**

Killer feature Framework 7

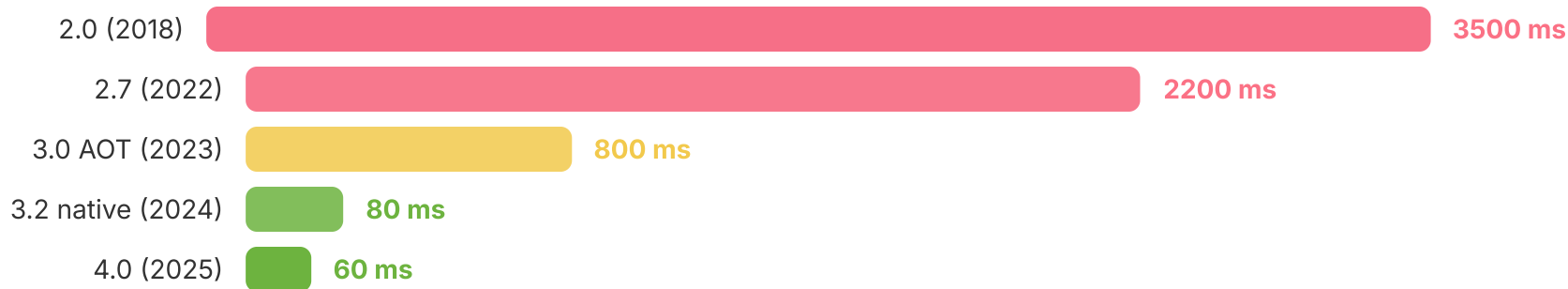
«This new release introduces automatic context propagation for Coroutines via the **PropagationContextElement** operator.»

— Spring Framework 7.0 Release Notes

Trace context (Sleuth/Micrometer Tracing), MDC, Security context — теперь автоматически переходят через `suspend` - границы. До 7.0 — терялись.

Cold start: эволюция в 50x

Spring Boot cold start (ms, hello-world)



AOT + GraalVM native сократили cold start в **50 раз**. CRaC в Boot 4 идёт дальше.

Порядковые значения по spring.io/blog (3.0 GA, Native, CRaC) и SpringOne 2024.



Финал

Три идеи, которые меняют всё



IoC

«Не ты создаёшь — тебе дают». Контейнер строит граф за вас — один раз, в startup.



Boot

«Convention over configuration». Тысячи дефолтов и один `application.yml` поверх.



Ecosystem

«На каждый домен — стартёр». Data, Security, Cloud, AI, Kafka — один флаг в deps.

Слова создателей

«Spring Boot 4 makes Spring development simpler, more modular, and ready for the next decade of Java and Kotlin applications.»

— Spring Boot 4.0 GA announcement, spring.io/blog, октябрь 2025

«Inversion of control isn't a feature — it's a way of writing software so that growth doesn't hurt.»

— перефраз идеи R. Johnson, *Expert One-on-One J2EE Design and Development*, 2003

Через 5 лет



Сегодня

```
fun main() { println("Hello") }
```



+1 год

ваш первый `@RestController` в проде



+3 года

свой `spring-boot-starter-*` в GitHub



+5 лет

вы контрибьютите в Spring Boot 6

Помните 60%? Теперь вы знаете, как ими стать.

Spring — это не фреймворк. Это привычка хорошо инженерить.

Спасибо!

Вопросы?

Напомню: 60%, 75k★, 10M/мес, 23 года — и теперь вы.

Лекция 6 курса «Kotlin» · СПбГУ МКН · 2026



Slidev · seriph · made with `kotlin` + `@SpringBootApplication`